Exp: 6

Date: 8/8/26 6. Error Detection and Correction using Hamming Code

Aim

To write a program to implement error detection and correction using hamming code.

ALGORITHM

1. Start the Program
2. Read For each 4-bit block ($d_1 d_2 d_3 d_4$), Compute even parity bits:

$P_1 = d_1 \oplus d_2 \oplus d_4$

$P_2 = d_1 \oplus d_3 \oplus d_4$ (Note: $\oplus$ represents X OR)

$P_4 = d_2 \oplus d_3 \oplus d_4$ Form the 7 bit codeword:

$P_1 P_2 d_1 P_4 d_2 d_3 d_4$.

3. Simulate Error: Introduce a Single-bit error at a user Specified position in the codeword.
4. Detect: At the receiver, Calculate the Syndrome bits (C_1, C_2, C_4) using the Same XOR logic on the received bits to find the Error Position ($C_4 C_2 C_1$ in binary)
5. Correct: If Error Position $\neq 0$, Invert the bit at that Specific position.
6. Extract the original 4 data bits from the Corrected Codeword.
7. Loop: Repeat Steps 2-6 for all n blocks
8. Stop

Program

```
def encode(d):
    """d: list of 4 bits [d1, d2, d3, d4] -> returns 7-bit
    hamming code list Positions (1-indexed): 1=P1 2=P2
    3=d1 4=P4 5=d2 6=d3 7=d4"""
    h = [0] * 7
    h[2] = d[0]  # Position 3
    h[4] = d[1]  # position 5
    h[5] = d[2]  # position 6
    h[6] = d[3]  # position 7
    h[0] = h[2] ^ h[4] ^ h[6]   P1 Covers 1,3,5,7
    h[1] = h[2] ^ h[5] ^ h[6]   P2 Covers 2,3,6,7
    h[3] = h[4] ^ h[5] ^ h[6]   P4 Covers 4,5,6,7
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos}
                -> Correcting...")
        h[error_pos-1] ^= 1
    else:
        print("No error detected")
    return error_pos
```

```
def to_str(bits):
    return "".join(str(b) for b in bits).

def main():
    print("=== HAMMING CODE - ERROR DETECTION AND
                        CORRECTION ===\n")
    n = int(input("Enter number of 4-bit data blocks
                        in the stream: "))

    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
        d = list(map(int, input("Enter 4 data bits (space
    separated, e.g. 1 0 1 1): ").split()))
        h = encode(d)
        print(f"Original Data Bits : {to_str(d)}")
        print(f"Encoded Hamming code (7 bits): {to_str(h)}")
        pos = int(input("Enter bit position to introduce
    error (1-7, 0 = no error): "))

        if 1 <= pos <= 7:
            h[pos-1] ^= 1
            print(f"Transmitted Code (with error induced
at Pos {pos}): {to_str(h)}")
        else:
            print(f"Transmitted Code (no error induced):
print(f"Received Code : {to_str(h)}")  {to_str(h)}")
    decode_and_correct(h)
print(f"Corrected Code : {to_str(h)}")
rd = [h[2], h[4], h[5], h[6]]
print(f"Recovered Data Bits : {to_str(rd)}")
```

OUTPUT

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1011): 1011
Original Data Bits     : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code          : 0100011
Checking for errors...
  Error detected at bit position: 3 -> Correcting...
Corrected Code         : 0110011
Recovered Data Bits    : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1011): 0110
Original Data Bits     : 0110
```

```
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code          : 1100100
Checking for errors...
  Error detected at bit position: 6 -> Correcting...
Corrected Code         : 1100110
Recovered Data Bits    : 0110
```

```
if __name__ == "__main__":
    main()
```



Result:

The program to implement error detection and correction using the hamming code concept was successfully written in python, executed and verified using a sample input data stream.